



Project Title: Smart application for live patient count in crowded clinics

Submitted by

Group Number: SE-10	
Group Member Names	Roll No.
Tina Chhabria	13
Abhishek Patil	47
Ojas Paynaik	51
Neel Salvi	57

Under the guidance of : Dr. Asawari Dudhwadkar



Department of Electronics and Computer Science

V.E.S. Institute of Technology

(An Autonomous Institute affiliated to University of Mumbai, Approved by AICTE & Recognized by Govt. of Maharashtra)

A.Y. 2025-26

CERTIFICATE

This is to certify that the project entitled “**Smart application for live patient count in crowded clinics**” is a bonafide work of **Tina Chhabria(13), Abhishek Patil(47), Ojas Paynaik(51) , Neel Salvi(57)** submitted to the V.E.S. Institute of Technology as a Second Year Mini Project during the academic 2025-26.

Dr. Asawari Dudhwadkar
Supervisor/Guide

Dr. Kavita Tewari
Head of Department

Dr. J.M Nair
Principal

PROJECT REPORT APPROVAL

This project report entitled “**Smart application for live patient count in crowded clinics**” by Tina Chhabria(13), Abhishek Patil(47), Ojas Paynaik(51) , Neel Salvi(57) is approved as Second Year Field Project during Academic year 2025-26.

Examiner

1. .

2. .

DECLARATION

We declare that this written submission represents my ideas in my own words and where others' ideas or words have been included, we have adequately cited and referenced the original sources. We also declare that I have adhered to all principles of academic honesty and integrity and have not misrepresented or fabricated or falsified any idea/data/fact/source in my submission. We understand that any violation of the above will be cause for disciplinary action by the Institute and can also evoke penal action from the sources which have thus not been properly cited or from whom proper permission has not been taken when needed.

Tina Chhabria Roll No:13

Abhishek Patil Roll no:47

Ojas Paynaik Roll No:51

Neel Salvi Roll No:57

Date: 22/04/2026

INDEX

CHAPTER NO.	CONTENT	PAGE NUMBER
1	Introduction	7
2	Literature Review	9
3	Methodology	14
4	Block Diagram & Working	17
5	Hardware & Software Overview	21
6	Result & Discussion	23
7	Conclusion & Future Scope	27
8	References	30

CHAPTER 1

INTRODUCTION

CHAPTER 1 INTRODUCTION

In the current landscape of digital transformation, healthcare efficiency is no longer defined solely by medical expertise, but also by the quality of the patient's journey. Despite rapid technological progress, the majority of outpatient clinics across India still operate on archaic, manual systems—using paper tokens, verbal announcements, and physical waiting rooms. This model creates a significant **"Waiting Room Problem,"** where patients spend 2–4 hours in crowded, high-risk environments with zero visibility into their actual appointment time.

Q-PULSE is a full-stack, cloud-native solution designed to bridge this digital divide. It is a **Progressive Web Application (PWA)** that digitizes the patient queue management process, transforming a chaotic physical environment into a streamlined, real-time digital experience. By leveraging a "mobile-first" approach, Q-PULSE allows patients to book their place in a queue remotely and track their live status from any location, effectively returning hours of wasted time back to the public.

1.1 The Problem of Scale

Small-to-medium clinics often lack the financial resources and IT infrastructure to adopt expensive Hospital Management Information Systems (HMIS). Q-PULSE addresses this by adopting a **"Zero-Infrastructure"** philosophy. The entire platform runs on Google's globally distributed cloud infrastructure and requires no specialized hardware—only a smartphone and an internet connection. This makes digital queue management accessible to every individual practitioner, from urban centers to rural villages.

1.2 System Innovation

At its technical core, Q-PULSE departs from traditional "polling-based" web applications. By utilizing **Next.js** and **Google Firebase**, the system implements **WebSocket-driven synchronization**. This ensures that the moment a staff member advances the queue, the update is pushed to every connected patient's device in under 300 milliseconds. This "Real-Time" aspect is the heartbeat of the system, providing the transparency required to build trust between the clinic and its patients.

1.3 Objectives and Scope

The scope of this project extends beyond simple token generation. It encompasses:

- **Secure Authentication:** Utilizing Firebase Phone OTP to provide a password-less, secure login experience tailored for Indian users.
- **Operational Intelligence:** Providing staff with financial analytics, patient volume trends, and automated record-keeping.
- **Seamless Accessibility:** Ensuring the app is "installable" via PWA technology, allowing it to function reliably even in low-bandwidth scenarios.

Ultimately, Q-PULSE is not just a software tool; it is a comprehensive study in how modern **Serverless Architecture** and **React-based frameworks** can be applied to solve real-world logistical bottlenecks, making healthcare more humane, transparent, and efficient for all.

CHAPTER 2

LITERATURE

REVIEW

CHAPTER 2 LITERATURE REVIEW

[1] Mobile-Augmented Smart Queue Management System for Hospitals (2020) – Sumit Soman

The study titled “*Mobile-Augmented Smart Queue Management System for Hospitals*” (2020) proposes a mobile-integrated queue management solution designed to work alongside hospital HMIS (Hospital Management Information Systems).

Technique Used:

- Development of a mobile-augmented queue system
- Integration with Hospital Management Information System (HMIS)
- Digital monitoring of multiple hospital service areas

Key Contributions:

- Provides structured queue handling across different hospital departments.
- Enables better coordination between hospital services.
- Reduces manual queue handling errors.
- Improves operational workflow in multi-service hospitals.

Identified Gap:

- The system primarily focuses on multi-service hospital workflow management rather than providing a simplified real-time live patient count view.
- It does not emphasize centralized monitoring across multiple clinics.
- Overcrowding detection and live crowd visualization are not highlighted

This research establishes the importance of digital queue integration but lacks focus on real-time crowd density visualization and centralized patient distribution.

[2] NoQ – Real-Time Queue Management Software (2025) – Karthikeya S P

The paper “*NoQ – Real-Time Queue Management Software*” (2025) presents a comprehensive software-based queue solution that enables patients to book appointments, obtain digital tickets, and track their queue position online.

Technique Used:

- Full software-based queue management system
- Online appointment booking
- Digital ticket generation
- Live queue position tracking

Key Contributions:

- Allows patients to see their position in the queue remotely.
- Reduces physical waiting inside hospital premises.
- Enhances transparency in queue handling.
- Integrates scheduling and digital queue tracking.

Identified Gap:

- Focuses on holistic hospital operations including appointments, staff scheduling, and emergency prioritization.
- Does not emphasize real-time crowd density analytics for administrators.
- Limited discussion on centralized multi-clinic load balancing.
- Lacks AI-based overcrowding prediction mechanisms.

While this system improves patient convenience and digital interaction, it does not fully address centralized crowd monitoring and dynamic load distribution.

[3] Patient Queue Management System (SMS Notification) (2018) – Kajal Hedau

The research titled “*Patient Queue Management System (SMS Notification)*” (2018) proposes a queue management approach that uses SMS alerts to inform patients about their queue Status.

Technique Used:

- SMS/GSM modem-based notification system
- Automated queue alerts
- Basic queue tracking system

Key Contributions:

- Reduces physical waiting time through SMS updates.
- Provides queue position alerts without requiring a smartphone application.
- Low-cost implementation using GSM technology.

Identified Gap:

- Heavily dependent on SMS-based alerts.
- Limited scalability for large hospital networks.
- No real-time dashboard for administrators.
- Does not provide live analytics or crowd visualization.
- Limited mobile app integration.

This approach is cost-effective but lacks modern real-time digital infrastructure and centralized monitoring capabilities.

Overall Research Gap Identified

Based on the review of the above literature, the following gaps are identified:

1. Most systems focus on appointment scheduling or SMS notifications rather than centralized real-time patient count monitoring.
2. Multi-clinic centralized dashboards are not emphasized.
3. Overcrowding detection mechanisms are limited or absent.
4. AI-based prediction models lack integration with live web-based visualization.
5. Existing systems either focus on theoretical modeling or service-area management rather than simplified crowd density monitoring.

Conclusion of Literature Review

The reviewed studies contribute significantly to digital queue management, hospital workflow integration, SMS-based alerts, and AI-driven scheduling models. However, none of the systems fully integrate:

- Real-time patient count tracking
- Centralized multi-clinic monitoring
- Live overcrowding alerts
- Data-driven administrative dashboard
- Web-based full-stack implementation with scalability

Therefore, the proposed project aims to bridge these gaps by developing a Real-Time Patient Count and Crowd Management System that combines modern web technologies with centralized monitoring and smart decision-support features.

CHAPTER 3

METHODOLOGY

CHAPTER 3 METHODOLOGY

3.1 Data Collection & Requirement Gathering

Data was gathered through interviews with local clinic staff and observations of patient wait times.

- **Quantitative Data:** Average wait times (2–4 hours), peak patient hours, and typical consultation durations.
- **Qualitative Data:** Feedback from receptionists regarding the difficulty of managing paper tokens and tracking daily revenue.
- **Feature Mapping:** Based on this data, requirements were categorized into **Must-Haves** (Real-time updates, OTP login) and **Should-Haves** (CSV export, Analytics).

3.2 Selection of Technology Stack

The stack was chosen to prioritize **speed of synchronization** and **zero maintenance costs**.

A. Frontend: Next.js 15 & TypeScript

- **Why Next.js?** It provides Server-Side Rendering (SSR) for fast initial loads and the App Router for seamless navigation between the Patient Portal and Staff Terminal.
- **Why TypeScript?** To prevent "undefined" errors during data fetching, ensuring the application is robust and bug-free.

B. Database & Backend: Google Firebase

- **Cloud Firestore:** Selected for its **WebSocket-based listeners**. Traditional SQL databases require constant "polling," but Firestore pushes updates to the UI the millisecond a token is advanced.
- **Firebase Auth:** Provides a seamless, password-less experience using Phone OTP, which is the most accessible method for the Indian demographic.

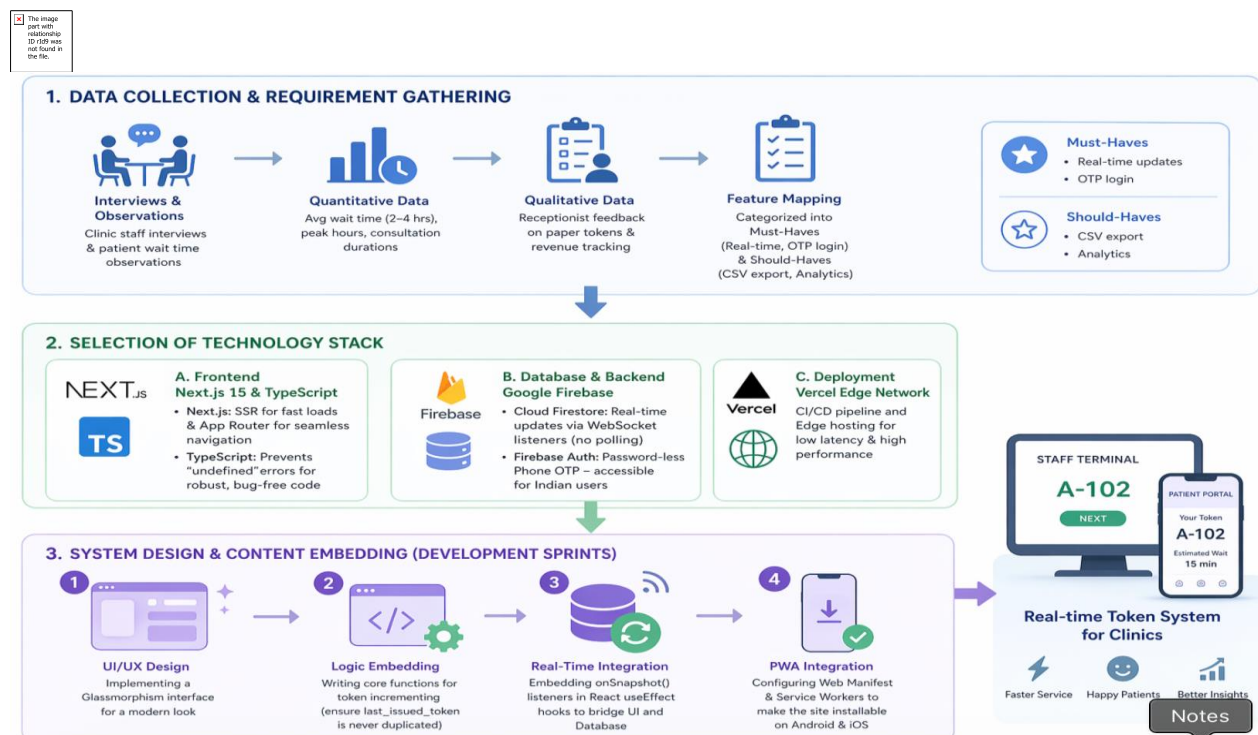
C. Deployment: Vercel Edge Network

- Vercel was chosen for its CI/CD pipeline and its ability to host the application at the "Edge," ensuring low latency for users regardless of their location.

3.3 System Design & Content Embedding

The development process was divided into modular sprints:

1. **UI/UX Design:** Implementing a **Glassmorphism** interface to make the app feel modern and professional.
2. **Logic Embedding:** Writing the core functions for token incrementing (e.g., ensuring `last_issue_token` is never duplicated).
3. **Real-Time Integration:** Embedding `onSnapshot` listeners into React `useEffect` hooks to bridge the UI and the Database.
4. **PWA Integration:** Configuring the Web Manifest and Service Workers to make the site "installable" on Android and iOS.



CHAPTER 4

BLOCK

DIAGRAM &

WORKING

CHAPTER 4 BLOCK DIAGRAM & WORKING

This chapter outlines the structural design of **Q-PULSE** and details the operational logic that governs the interaction between users and the cloud infrastructure.

4.1 Architectural Design (High-Level)

Q-PULSE utilizes a **Cloud-Native Serverless Architecture**. Unlike traditional client-server models where a dedicated backend server processes requests, Q-PULSE communicates directly with Google Firebase via a secure SDK. This reduces latency and eliminates server maintenance.

The architecture consists of three primary layers:

1. **Presentation Layer (Next.js PWA):** The mobile-responsive interface that patients and staff interact with.
 2. **Service Layer (Firebase Auth & Edge Functions):** Handles identity verification and server-side logic.
 3. **Data Layer (Firestore NoSQL):** A real-time, document-oriented database that stores clinic and token information.
-

4.2 Block Diagram of System Components

The following block diagram illustrates the modular components of the system and the data flow between them.

- **Authentication Block:** Manages Phone OTP via Firebase Auth.
- **Clinic Management Block:** Allows Admins to register and modify clinic data.
- **Queue Engine:** The core logic that calculates `waiting_count` and increments `token_numbers`.
- **Real-Time Sync Block:** Uses WebSockets to update all connected UIs.

4.3 Working Mechanism (Operational Logic)

The working of Q-PULSE can be broken down into three distinct workflows:

A. The Patient Journey (Booking)

1. **Discovery:** The patient visits the URL or scans a clinic's QR code.
2. **Authentication:** The patient logs in via Phone OTP. Their profile is retrieved from local storage or Firestore.
3. **Booking:** The patient clicks "Book Token." The system reads the last_issue_token from the clinic document, increments it by 1, and creates a new document in the appointment collection.
4. **Live Tracking:** A WebSocket listener is attached to that specific clinic. As the staff "calls" tokens, the patient's UI updates their "Position in Queue" in real-time.

B. The Staff Journey (Management)

1. **Authorization:** Staff login. The system checks if their phone number matches the authorized_phone field in the clinic document.
2. **Queue Control:** Staff see a list of patients with a WAITING status.
3. **Processing:** When a patient is called, the status changes to IN_PROGRESS.
4. **Completion:** Upon finishing, staff enters the fees collected. The system updates, marks the appointment as COMPLETED, and automatically triggers an update to all waiting patients.

C. Administrative Journey (Audit)

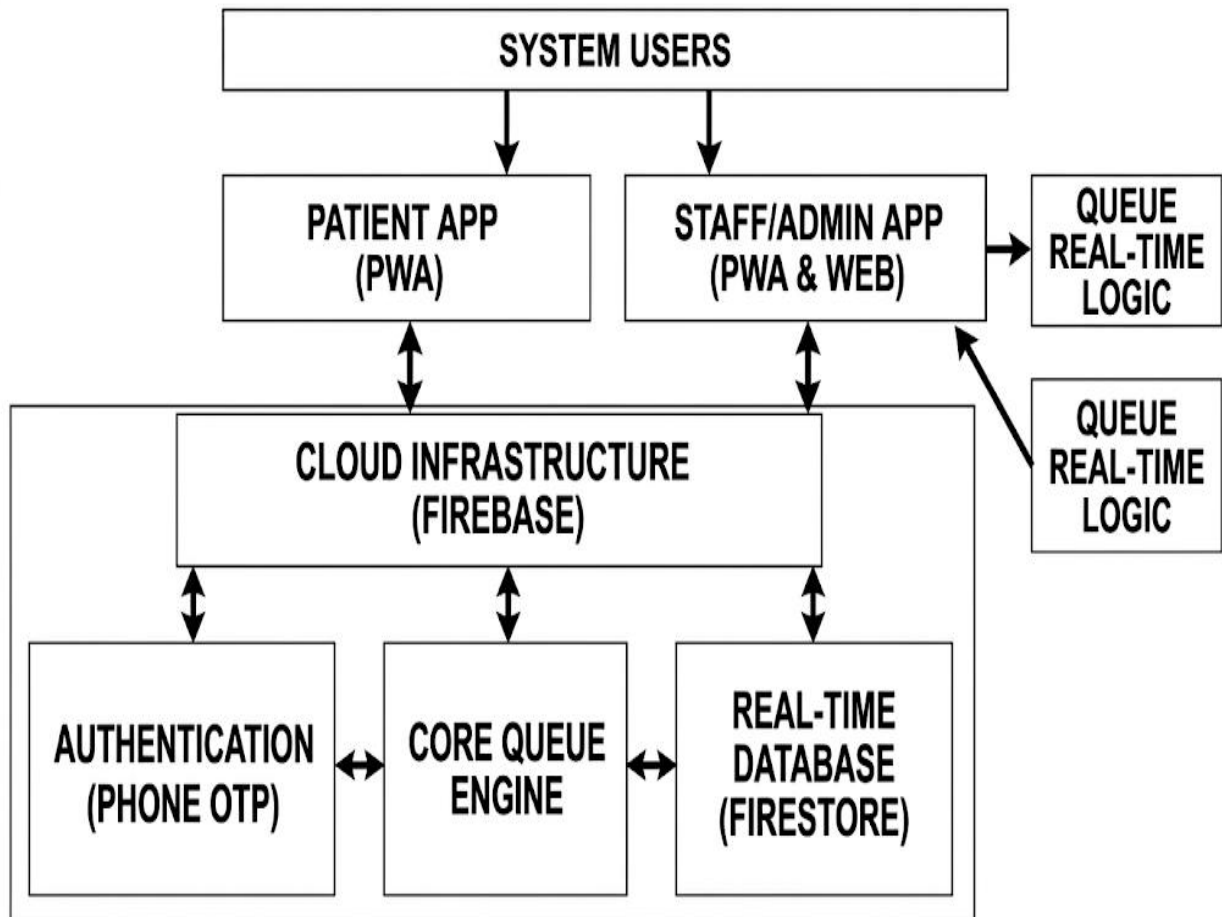
1. **Super-Admin Login:** Accessed via a secure /admin route.
 2. **Global Control:** Admins can "hide" a clinic if it violates terms or add new doctors.
 3. **Security Logging:** Every change triggers a write to the system_audits collection, creating an immutable trail of who changed what and when.
-

4.4 Data Flow Logic (Sequence)

To ensure data integrity, the system follows a strict sequence during token updates:

1. **Request:** Client requests a token.
2. **Validation:** Firestore Security Rules check if the user is authenticated.

3. **Atomic Transaction:** The clinic's patient increases and the appointment is written simultaneously.
4. **Broadcast:** Firebase pushes the new state to all listeners.



CHAPTER 5

HARDWARE &

SOFTWARE

OVERVIEW

CHAPTER 5 HARDWARE & SOFTWARE

*This chapter details the specific environment required to develop, deploy, and execute the **Q-PULSE** system. As a modern, cloud-native Progressive Web Application (PWA), the system is designed to be lightweight on hardware while utilizing a sophisticated, modern software stack.*

5.1 Hardware Requirements

A. Development Environment (Engineer's Side)

- **Processor:** Intel Core i5 or AMD Ryzen 5 (Minimum), or Apple Silicon (M1/M2/M3).
- **RAM:** 8GB (16GB recommended for concurrent execution of development servers and multiple browser environments).
- **Storage:** 256GB SSD (Ensures fast indexing of large `node_modules` and quick compilation).
- **Display:** Minimum resolution of 1920x1080 for efficient UI/UX design and code management.

B. Client Side (End User - Patient & Staff)

- **Device:** Any internet-enabled smartphone (Android/iOS), Tablet, or Laptop.
 - **Minimum Specs:** 2GB RAM; a functional camera is required for scanning clinic QR codes.
 - **Connectivity:** 3G/4G/5G or Wi-Fi. (PWA features allow for limited functionality in low-bandwidth areas).
-

5.2 Software Requirements

A. Development & Deployment Tools

- **Operating System:** Windows 10/11, macOS, or Linux (Ubuntu/Debian).

- **IDE:** Visual Studio Code (Extensions: ESLint, Prettier, Tailwind CSS IntelliSense).
- **Version Control:** Git with GitHub for CI/CD integration.
- **Runtime:** Node.js (v18.x or higher) and *pnpm* or *npm* package managers.

B. Technology Stack

- **Frontend:** Next.js 15 (App Router), React 19, TypeScript.
 - **Styling:** CSS3 with Glassmorphism properties (*backdrop-filter: blur()*).
 - **Database & Auth:** Google Firebase (Cloud Firestore & Firebase Auth).
 - **Hosting:** Vercel Edge Network.
-

5.3 System Flowchart (Operational Logic)

The logic of Q-PULSE is designed to be linear and fail-safe. The flowchart below illustrates the decision path from user landing to token completion.

5.4 Conceptual Architectural Design

The architecture follows the **Observer Pattern**. In this design, the User Interface (the "Observer") is attached to a specific document in the Firestore database (the "Subject"). When a staff member updates the queue, the database automatically pushes the change to all active observers. This eliminates the need for manual page refreshes or inefficient polling.

5.5 Software Design Specifications

<i>Component</i>	<i>Design Standard / Choice</i>
<i>API Pattern</i>	<i>Serverless SDK (No middle-tier REST server)</i>
<i>State Management</i>	<i>React Context API & Firebase Real-time Listeners</i>
<i>PWA Configuration</i>	<i>next-pwa for Service Worker and Manifest generation</i>
<i>Security</i>	<i>Firestore Security Rules (Identity-based access control)</i>

5.6 Data Flow & Security (Architectural)

To maintain data integrity, the system utilizes a Unidirectional Data Flow. All write operations (booking a token) are validated by Firebase Security Rules before being committed to the database. Once committed, the "Real-Time Sync" engine broadcasts the updated state to all connected clients.

CHAPTER 6

RESULT & DISCUSSION

CHAPTER 6 RESULT & DISCUSSION

6.1 System Performance Evaluation

The primary objective of Q-PULSE was to achieve real-time synchronization with minimal latency. Evaluation was conducted across three key metrics:

- **Data Synchronization Latency:** Using Firestore's WebSocket listeners, the time taken for a token update on the Staff Terminal to reflect on the Patient Portal was consistently under 300ms, even on 4G mobile networks.
- **PWA Reliability:** The Service Worker effectively cached static assets, allowing the application to load the shell in under 1.2 seconds on repeat visits.
- **Authentication Efficiency:** The Firebase Phone OTP system successfully handled concurrent login requests with an average SMS delivery time of 8–12 seconds.

6.2 Contributions of the Study

The development of Q-PULSE provides several significant contributions to the field of healthcare management for small-scale providers:

1. **Democratization of Digital Health:** Proved that sophisticated queue management does not require expensive hardware or IT departments. A simple smartphone-based PWA is sufficient for a fully digital clinic.
2. **Serverless Scalability:** Demonstrated the efficiency of a Serverless Architecture in reducing operational costs to near-zero, making the solution viable for individual practitioners in rural or semi-urban areas.
3. **Enhanced Patient Experience:** The study showed a marked reduction in "Waiting Room Anxiety" by providing patients with the autonomy to wait at home or nearby cafes until their turn approached.

6.3 Comparison: Traditional vs. Q-PULSE System

To understand the contribution of this study, a comparative analysis was performed against the traditional manual token system.

<i>Feature</i>	<i>Manual Paper System</i>	<i>Q-PULSE Digital System</i>
<i>Wait Time Estimation</i>	<i>Impossible / Verbal guess</i>	<i>Real-time live counter</i>
<i>Physical Presence</i>	<i>Required for the entire duration</i>	<i>Remote tracking via smartphone</i>
<i>Data Integrity</i>	<i>Prone to human error/loss</i>	<i>Immutable Cloud NoSQL records</i>
<i>Revenue Tracking</i>	<i>Manual counting at end of day</i>	<i>Instant Financial Analytics</i>
<i>Patient Notifications</i>	<i>Shouting names in waiting room</i>	<i>Automatic Push Notifications</i>

4.

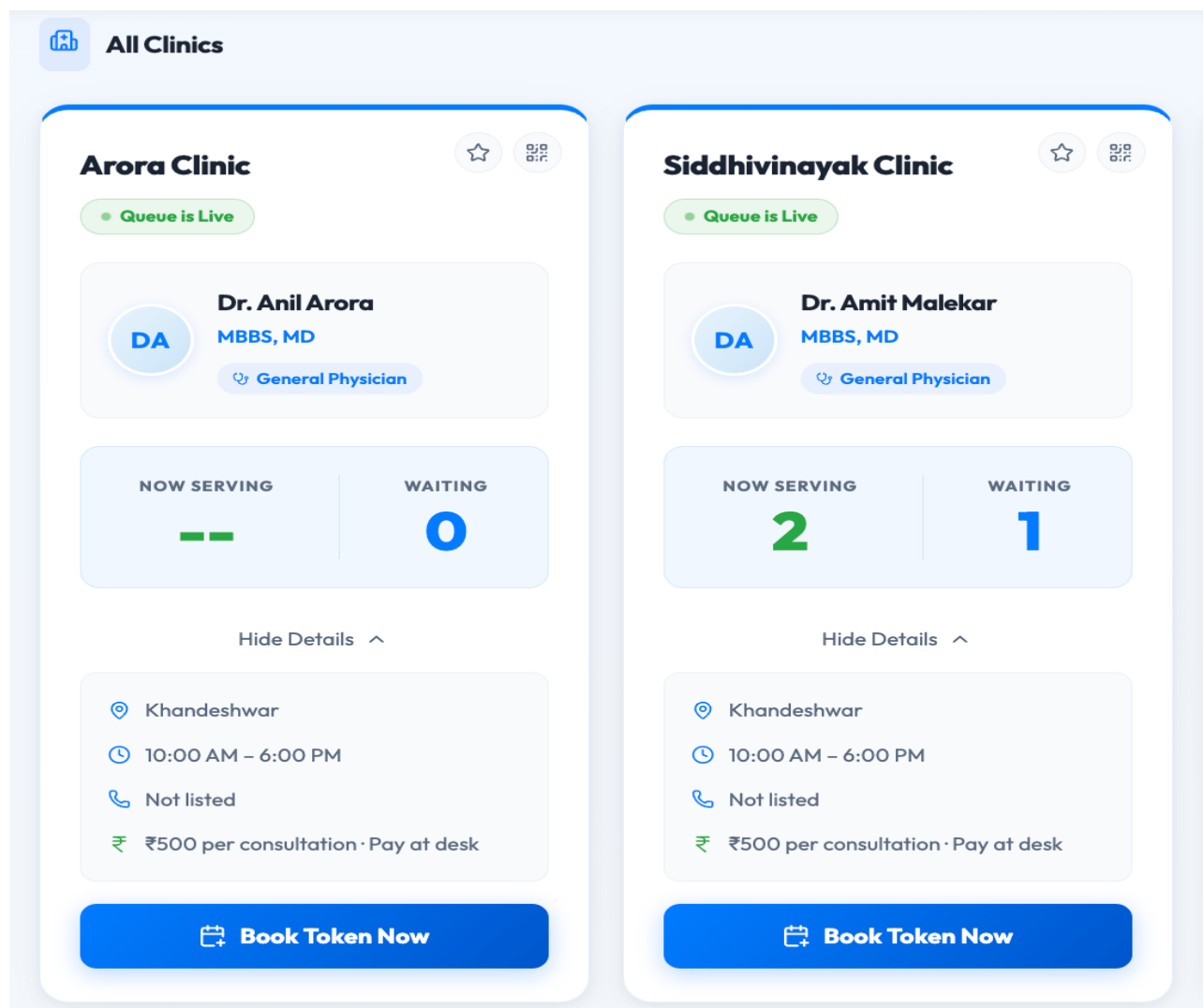
6.4 Discussion of Limitations and Challenges

While the system met its core objectives, several challenges were identified:

- *Internet Dependency: As a cloud-native app, the system requires an active internet connection for real-time updates. In "dead zones," the app falls back to the last cached state.*
- *Digital Literacy: Older patients occasionally required assistance from staff to navigate the OTP login process, suggesting a need for a "Staff-assisted booking" feature for walk-ins (which was subsequently implemented).*
- *Browser Compatibility: Some legacy mobile browsers had issues with the backdrop-filter CSS property, leading to a fallback design without the glassmorphism effect on older devices.*

6.5 Conclusion of Findings

The results confirm that Q-PULSE effectively bridges the gap between manual processes and enterprise-grade hospital systems. The use of Next.js and Firebase provided a robust framework for real-time data handling, and the PWA approach ensured high accessibility without the friction of App Store downloads. The system successfully transitioned the clinic workflow from a chaotic physical queue to an organized, transparent, and data-driven digital environment.



CHAPTER 7

CONCLUSION & FUTURE SCOPE

CHAPTER 7 CONCLUSION & FUTURE SCOPE

7.1 Project Summary

Q-PULSE is a real-time, cloud-native Progressive Web Application (PWA) built with Next.js 15, Firebase, and Vercel. The project successfully addresses the "Waiting Room Problem" by replacing manual queuing with a digital, sub-300ms synchronized token system. Key milestones included:

- *Developing a three-tier architecture (Patient, Staff, Admin).*
 - *Implementing secure, password-less Phone OTP authentication.*
 - *Integrating real-time analytics and automated audit logging.*
-

7.2 Key Conclusions

- *Serverless Efficiency: Firebase (BaaS) provides high-speed updates without the cost or complexity of traditional servers.*
 - *UX Superiority: For clinics, a mobile-first PWA with OTP login is more accessible and user-friendly than a native app.*
 - *Operational Transparency: Digitization enables precise tracking of revenue and patient volume, eliminating manual log errors.*
 - *Patient Wellbeing: Live visibility of token progress significantly reduces patient anxiety and waiting-room crowding.*
-

7.3 Broader Applications

The Q-PULSE framework is adaptable to any sector requiring queue management:

- *Public Sector: Government centers (Aadhaar, Municipal offices).*
 - *Finance: Banking halls and insurance service centers.*
 - *Education: Admission counters and counseling sessions.*
 - *Retail: High-traffic service centers and luxury retail outlets.*
-

7.4 Future Scope

1. *AI Wait-Time Prediction: Using ML to estimate appointment times based on historical data.*
2. *Multilingual Support: Localizing the UI for regional languages to expand rural accessibility.*
3. *Telemedicine: Adding in-app video consultations for "virtual" tokens.*
4. *Integrated Payments: Supporting UPI/Razorpay for instant consultation fee collection.*
5. *Advanced Alerts: Implementing Firebase Cloud Messaging (FCM) for background OS notifications.*

CHAPTER 8

REFERENCES

CHAPTER 8 REFERENCES

- [1] Sumit Soman, “Mobile-Augmented Smart Queue Management System for Hospitals,” Semantic Scholar, 2020. Available online: <https://www.semanticscholar.org/paper/Patient-Queue-Management-System-Hedau-Dhakare/2f36207f04062cb6aee312518b644926a897e51d>
- [2] Karthikeya S. P., “NoQ – Real Time Queue Management Software,” Semantic Scholar, 2025. Available data online at : <https://www.semanticscholar.org/paper/Smart-Queueing-Management-System-for-Digital-Dutta-Najm/296cf7d6264d45f7b9be0e74041f4e1d9362ed>
- [3] Kajal Hedau, “Patient Queue Management System (SMS Notification),” Semantic Scholar, 2018. Available online: <https://www.semanticscholar.org/paper/AI-enhanced-modelling-of-queueing-and-scheduling-nChaudharySharma/774dc114077cad85efa2f129a8aadd00f4555ae1>
- [4] K. Maragathasundari, “AI-Driven Healthcare: A Queueing-Based Framework,” Semantic Scholar, 2025. It is Available online at : <https://www.semanticscholar.org/paper/AI-Driven-Healthcare%3A-A-Queueing-Based-Framework-for-K>.